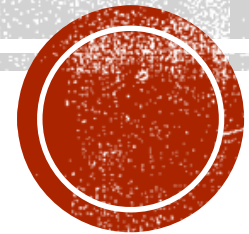


RECONFIGURABLE COMPUTING

CS G553: Reconfigurable computing

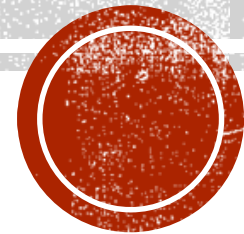
Dr. Meetha V Shenoy

Lect-11



ARCHITECTURAL SYNTHESIS (HIGH LEVEL SYNTHESIS)

SCHEDULING & TEMPORAL PARTITIONING



SCHEDULING

- Inputs:
 - A DFG
 - An architecture (i.e. a set of processing elements)
- Output:
 - Starting time of each node on a given resource



UNCONSTRAINED SCHEDULING

- **Assumption:**
 - unlimited amount of resources
 - Usually as a pre-processing step for other algorithms
 - E.g. computation of the upper and lower bounds on the starting time of operations.
 - o Lower bound: the earliest time a module can be scheduled,
 - o Upper bound: the latest time at which a module can be started.



UNCONSTRAINED SCHEDULING

ASAP(as soon as possible)

- Defines the earliest starting time for each node in the DFG
- Computes a minimal latency

ALAP(as late as possible)

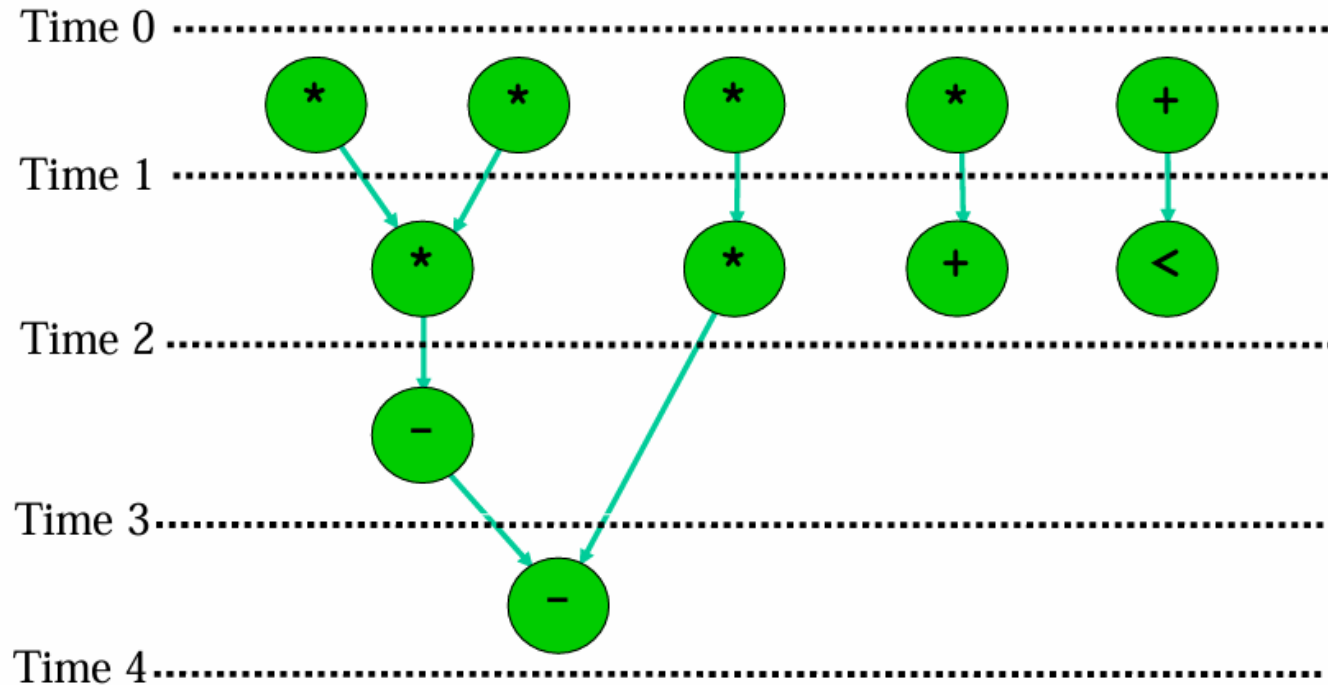
- Defines the latest starting time for each node in the DFG according to a given latency



ASAP EXAMPLE

Assume a latency of one clock cycle for an operation

- Unconstrained scheduling with optimal latency : $L = 4$



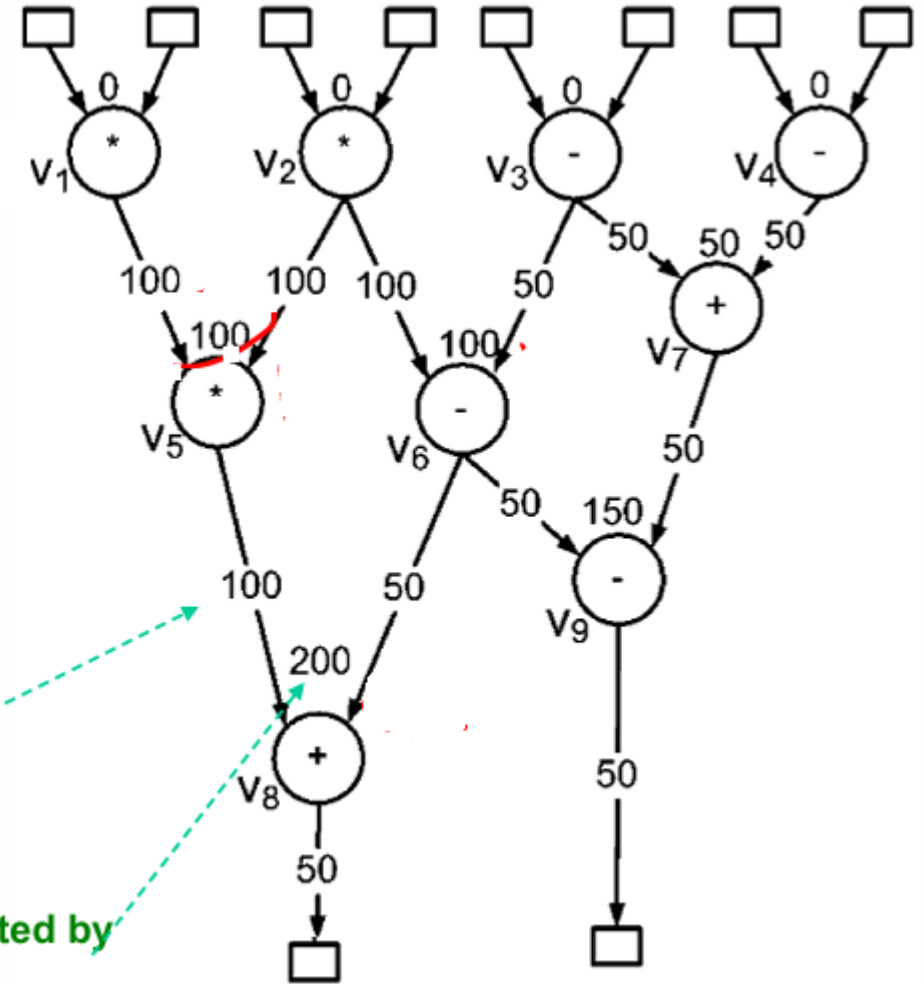
ASAP EXAMPLE

- Assumptions:

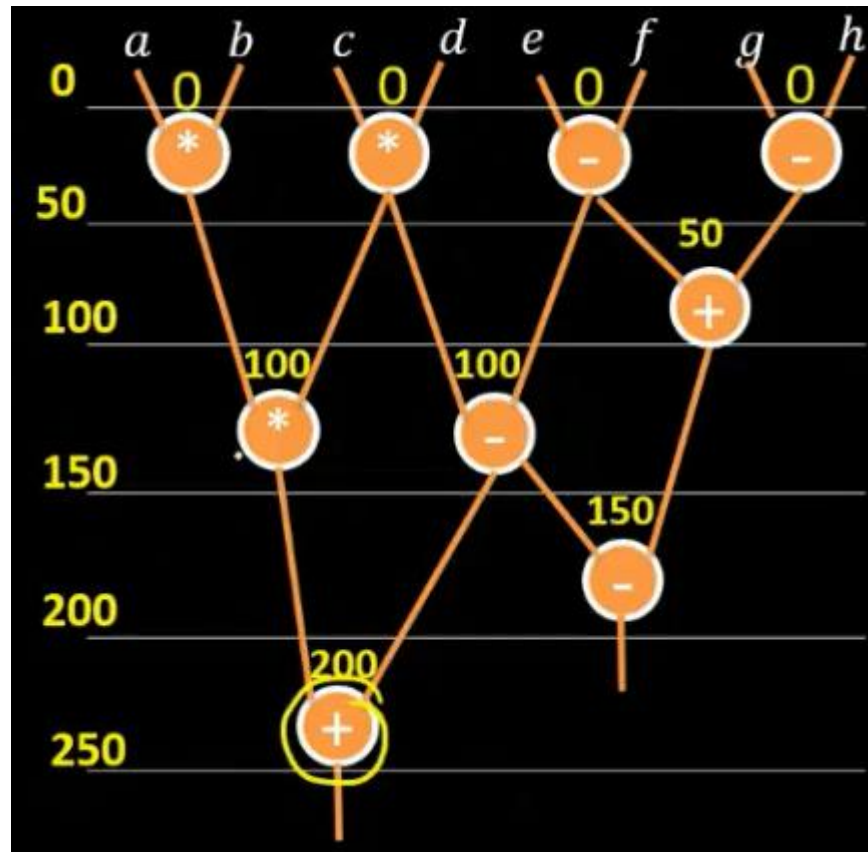
- o Multiplication: latency of 100 clocks,
- o Addition/subtraction: 50 clocks,
- o data transmission delay is neglected.

Computation delay of
the prev. node

Node's starting time as computed by
the algorithm.



ASAP EXAMPLE

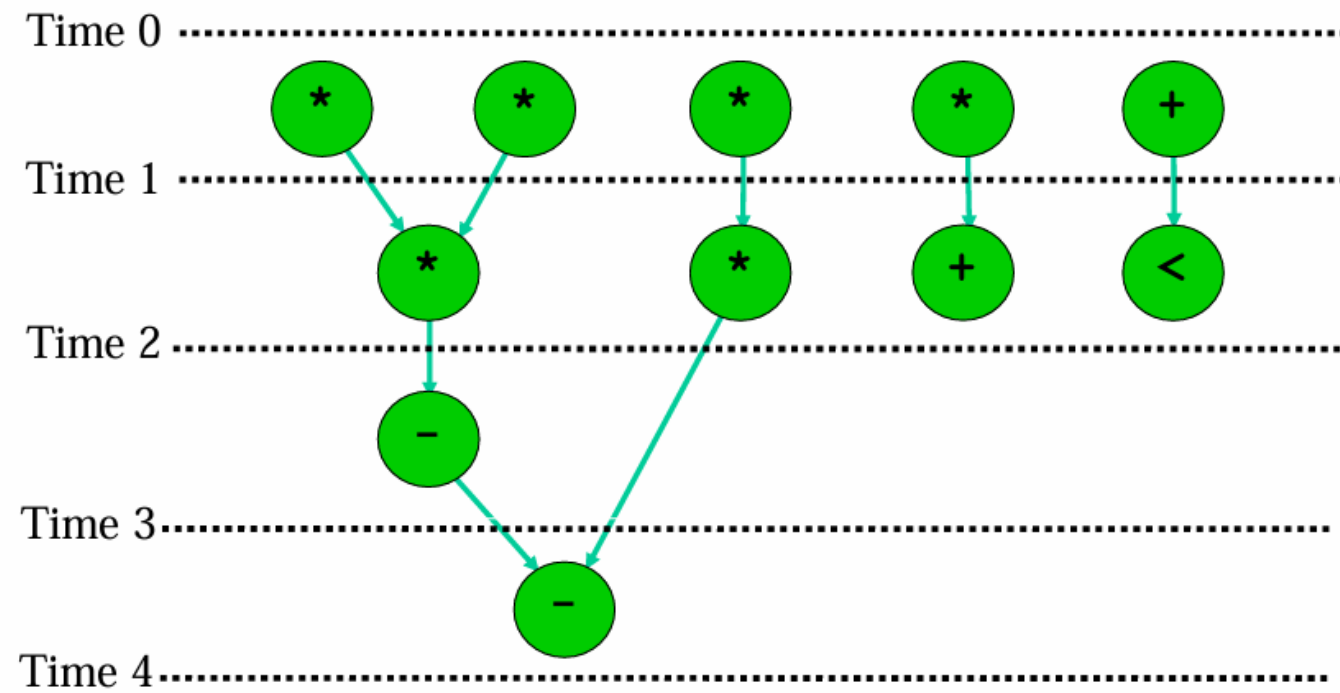


ASAP ALGORITHM

Algorithm: ASAP-Algorithm for unconstrained scheduling

```
1: for each node  $v \in V$  do
2:   if  $v$  has no predecessors then
3:      $\zeta(v) := 0$ 
4:      $V := V - v$ 
5:   end if
6: end for
7: while  $V \neq \emptyset$  do
8:   select a vertex  $v_i \in V$  whose predecessors are all scheduled
9:   schedule  $v_i$  by setting  $\zeta(v_i) := \max_{(v_j, v_i) \in E} (\zeta(v_j) + t_j)$ 
10:   $V := V - v_i$ 
11: end while
```

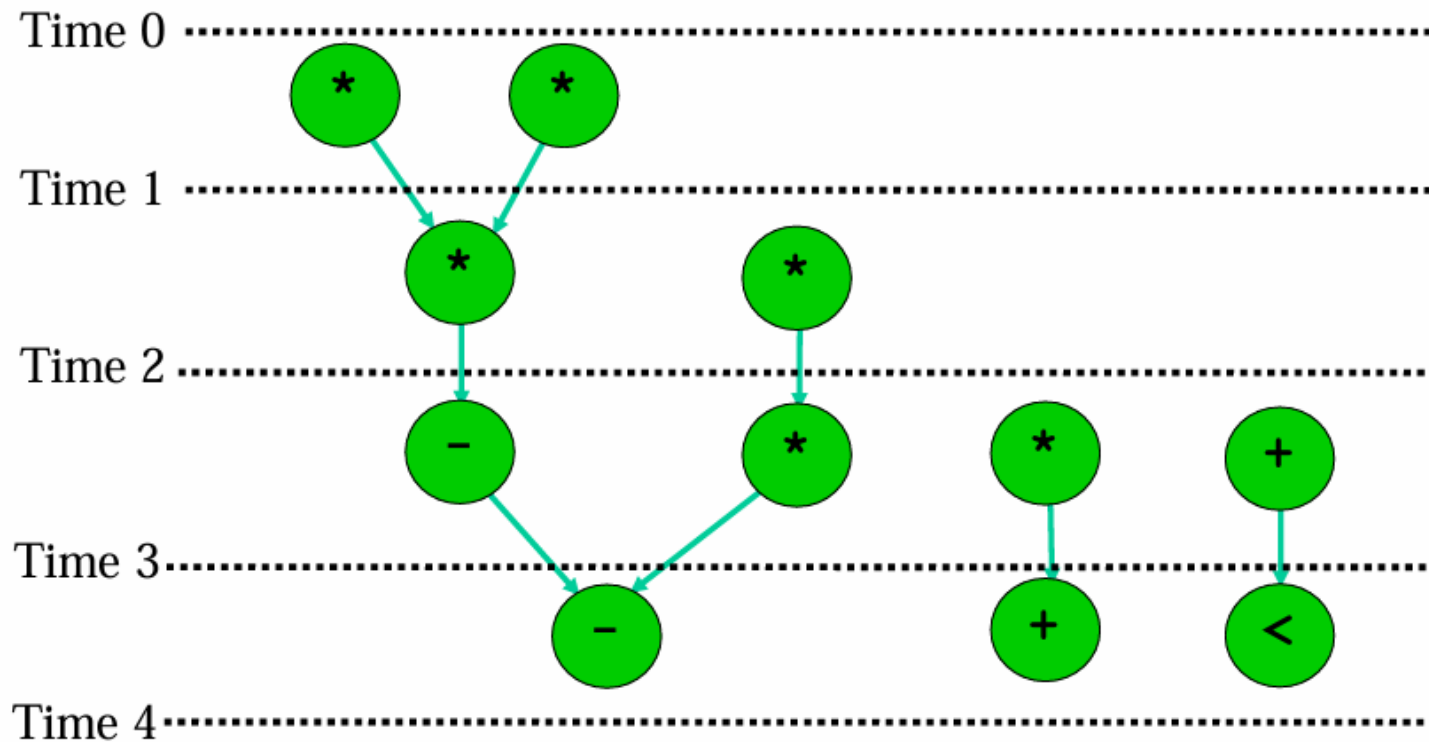




ALAP EXAMPLE

- Assume a latency of one clock cycle for an operation

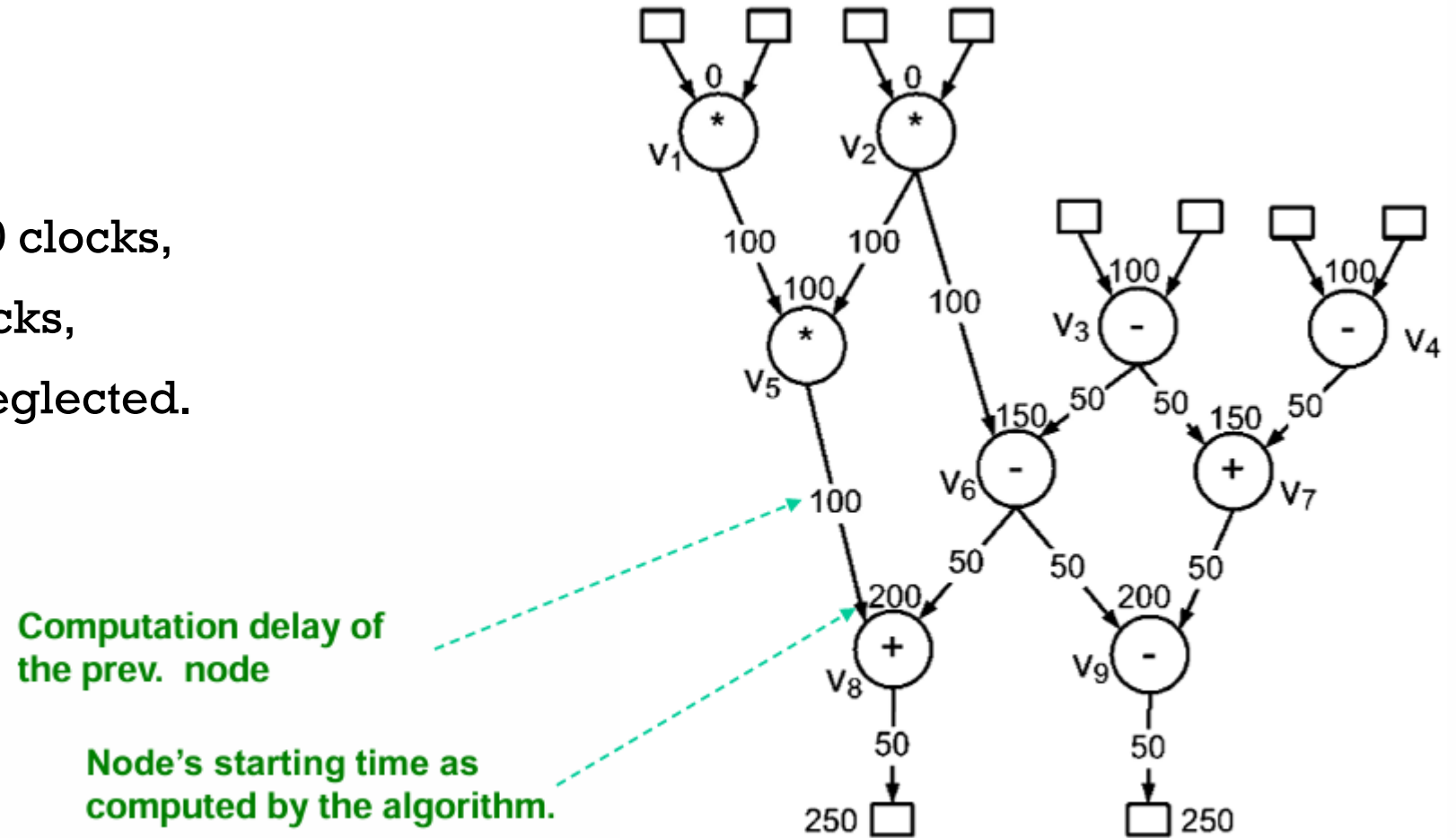
- Unconstrained scheduling with optimal latency :
 $L = 4$



ALAP EXAMPLE

- Assumptions:

- o Multiplication: latency of 100 clocks,
- o Addition/subtraction: 50 clocks,
- o data transmission delay is neglected.

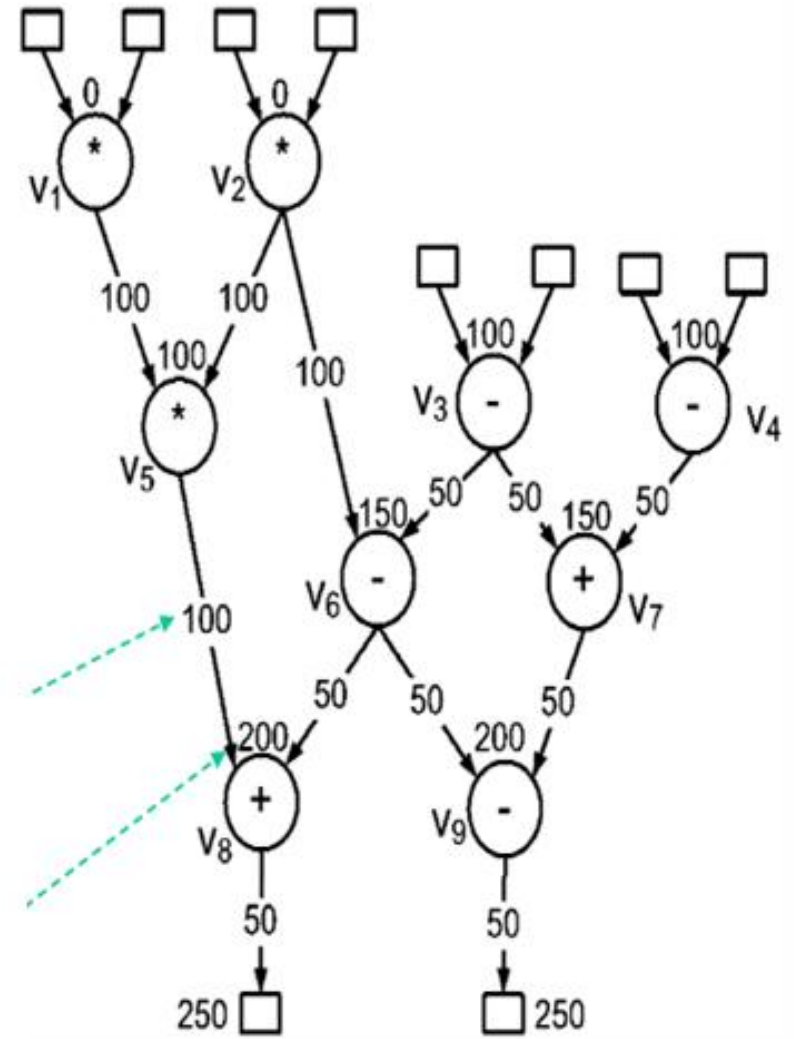
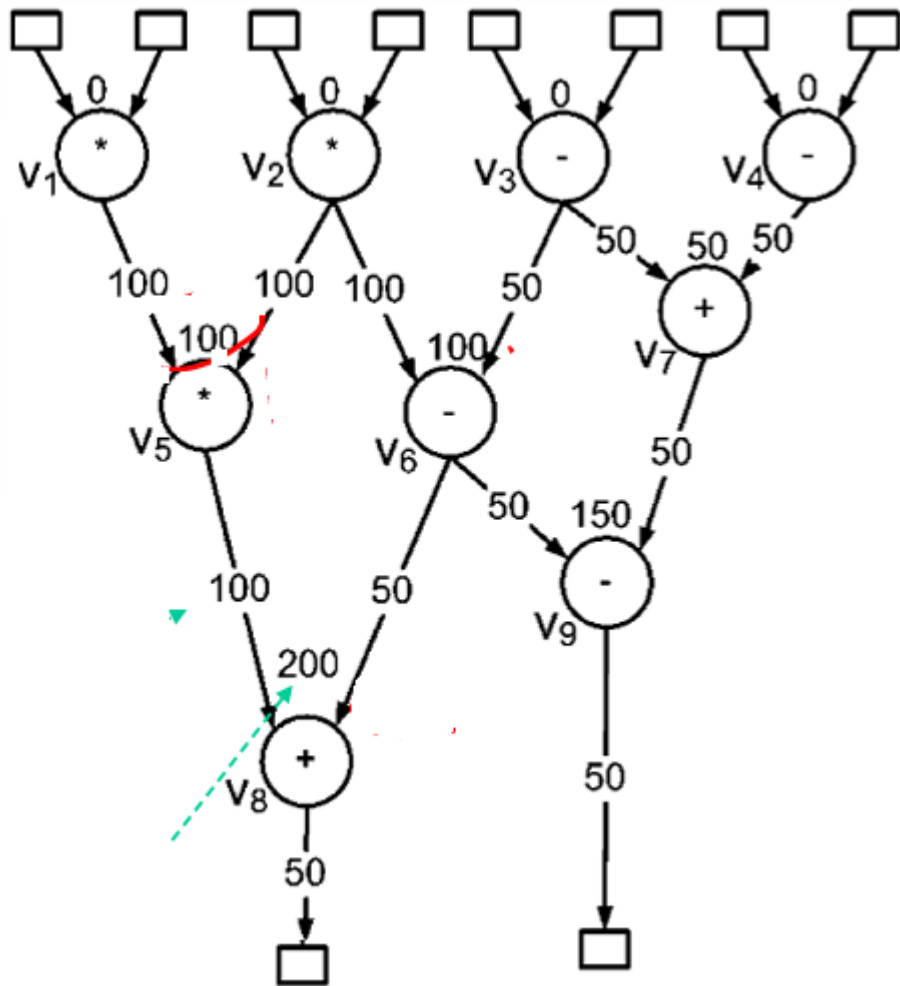


ALAP ALGORITHM

Algorithm: ALAP-Algorithm for unconstrained scheduling

```
1: for each node  $v \in V$  do
2:   if  $v$  has no successors then
3:      $\zeta(v) := \lambda$ 
4:      $V := V - v$ 
5:   end if
6: end for
7: while  $V \neq \emptyset$  do
8:   select a vertex  $v_i \in V$  whose successors are all scheduled
9:   schedule  $v_i$  by setting  $\zeta(v_i) := \max_{(v_j, v_i) \in E} (\zeta(v_j) - t_i)$ 
10:   $V := V - v_i$ 
11: end while
```





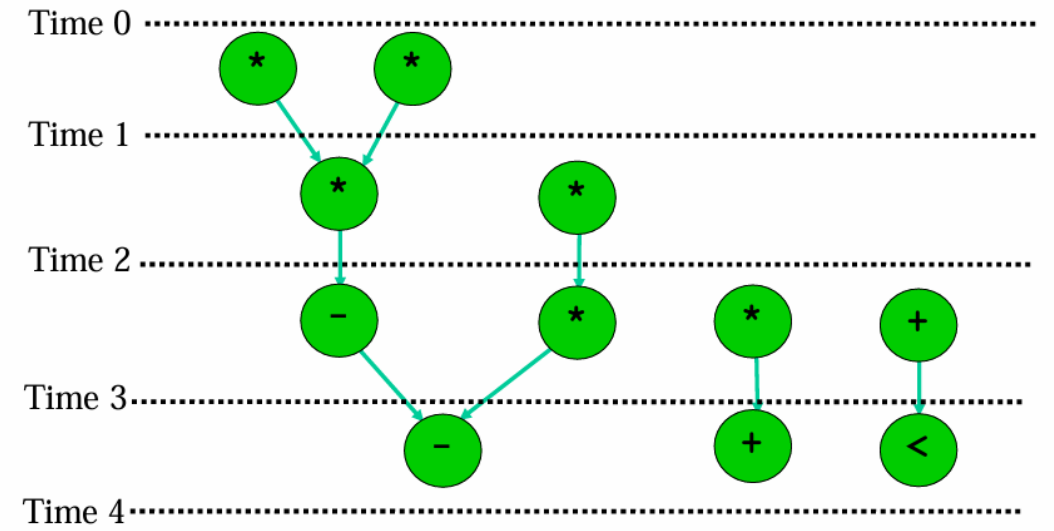
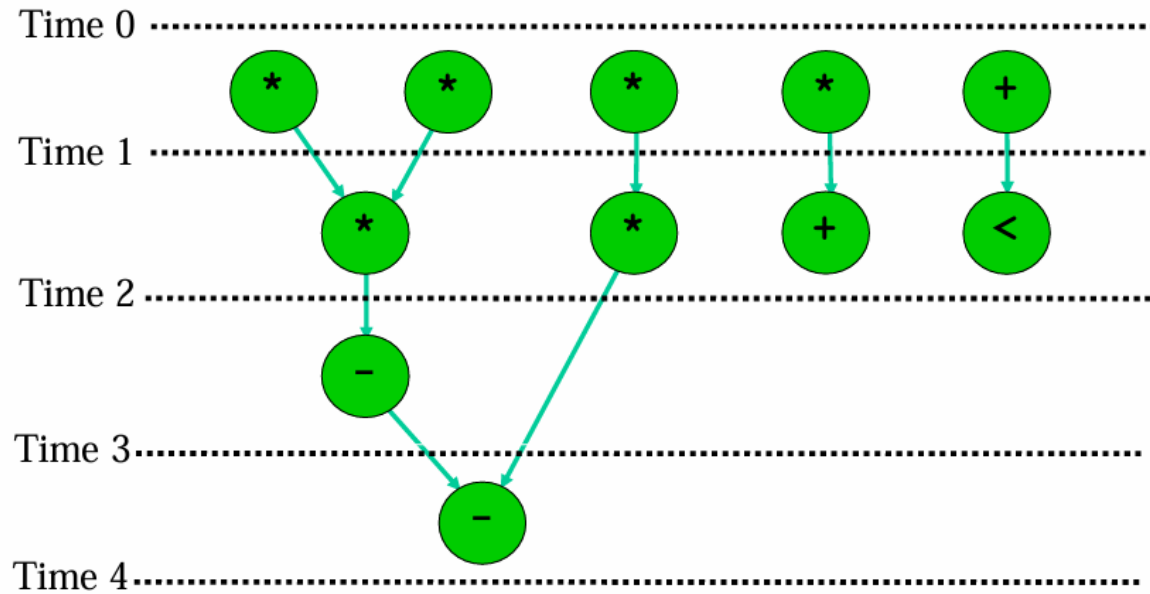
MOBILITY

The **mobility** of a node: (ALAP starting time) – (ASAP starting time)

Mobility = 0 → node is on a critical path



MOBILITY



CONSTRAINED SCHEDULING

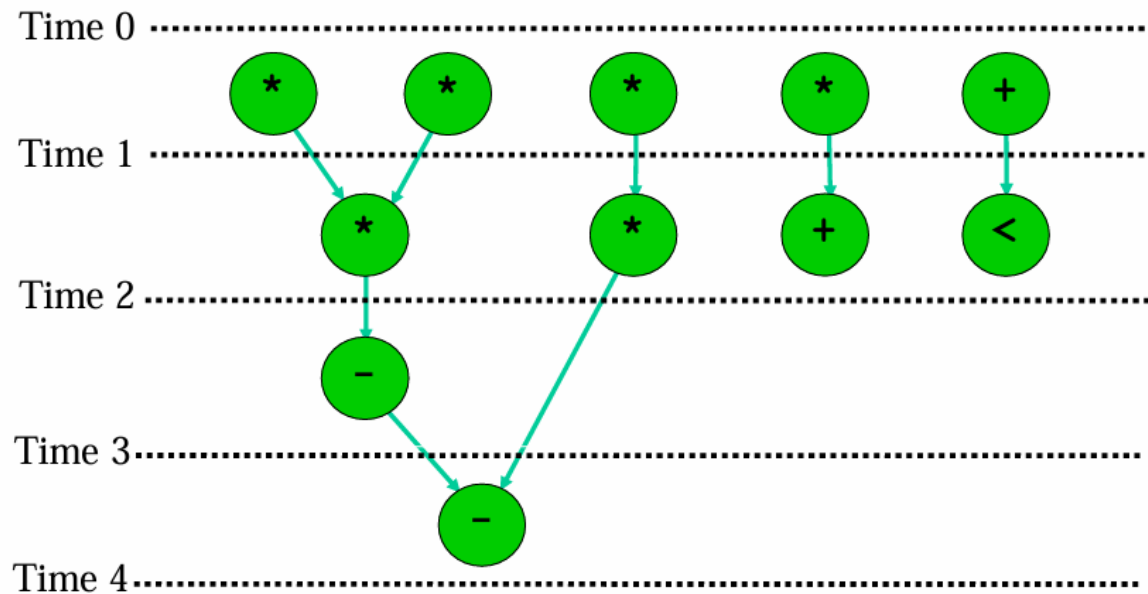
- o A set of fixed resources available (very critical for ASIC).
- o Many tasks competing for a given resource, • → One of them must be chosen according to a given criteria and the rest will be scheduled later.

Extended ASAP, ALAP:

- o Compute ASAP or ALAP
- o Assign the tasks earlier (ASAP) or later (ALAP), until the resource constraints (e.g. area) are fulfilled

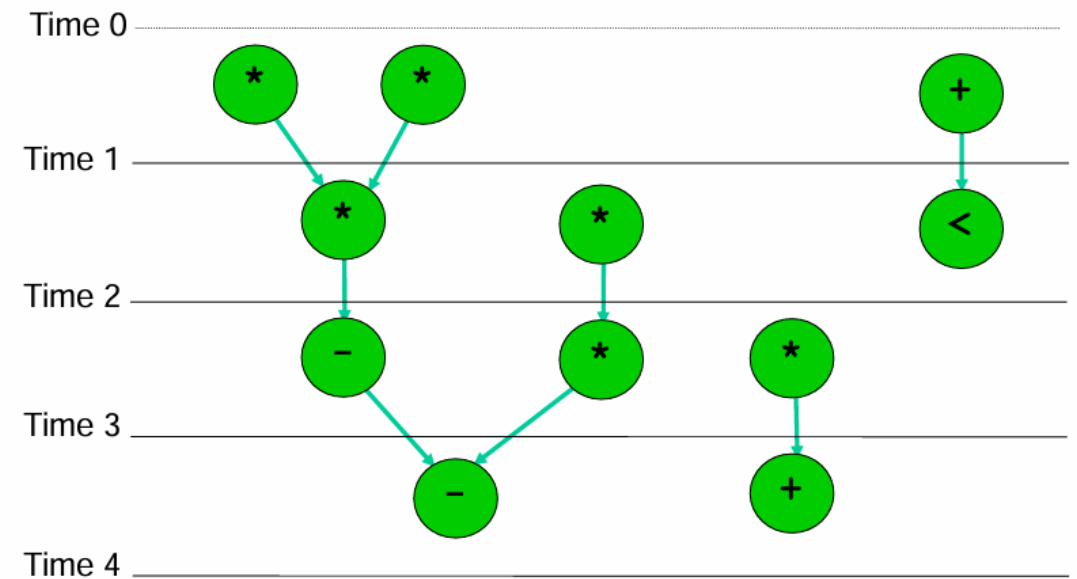
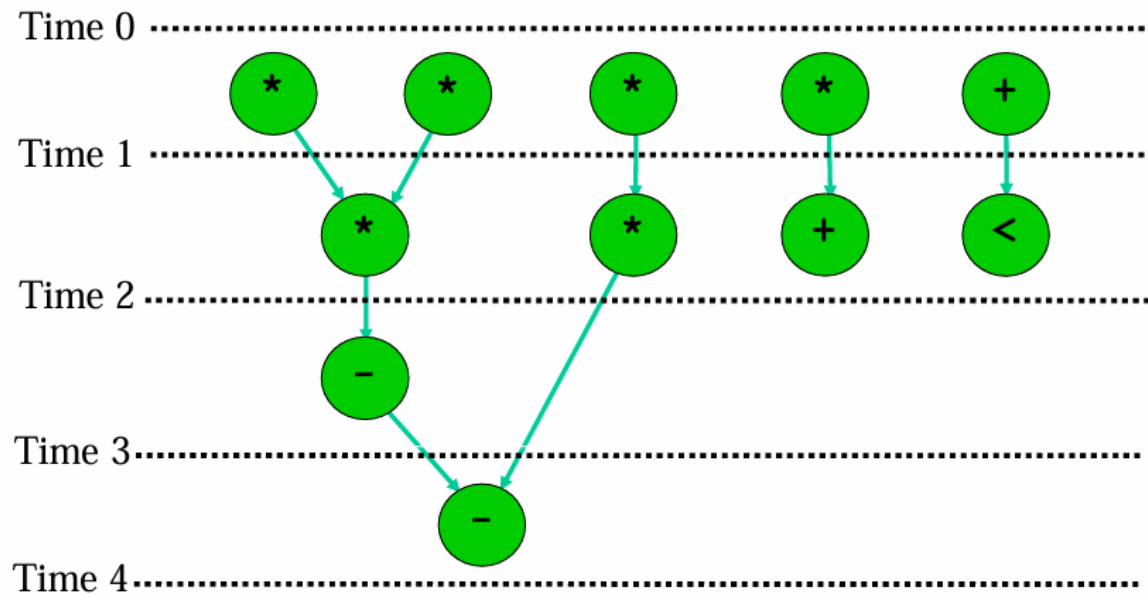


- 2 Multipliers, 2 ALUs (+, −, <)



EXTENDED ASAP

- 2 Multipliers, 2 ALUs (+, −, <)



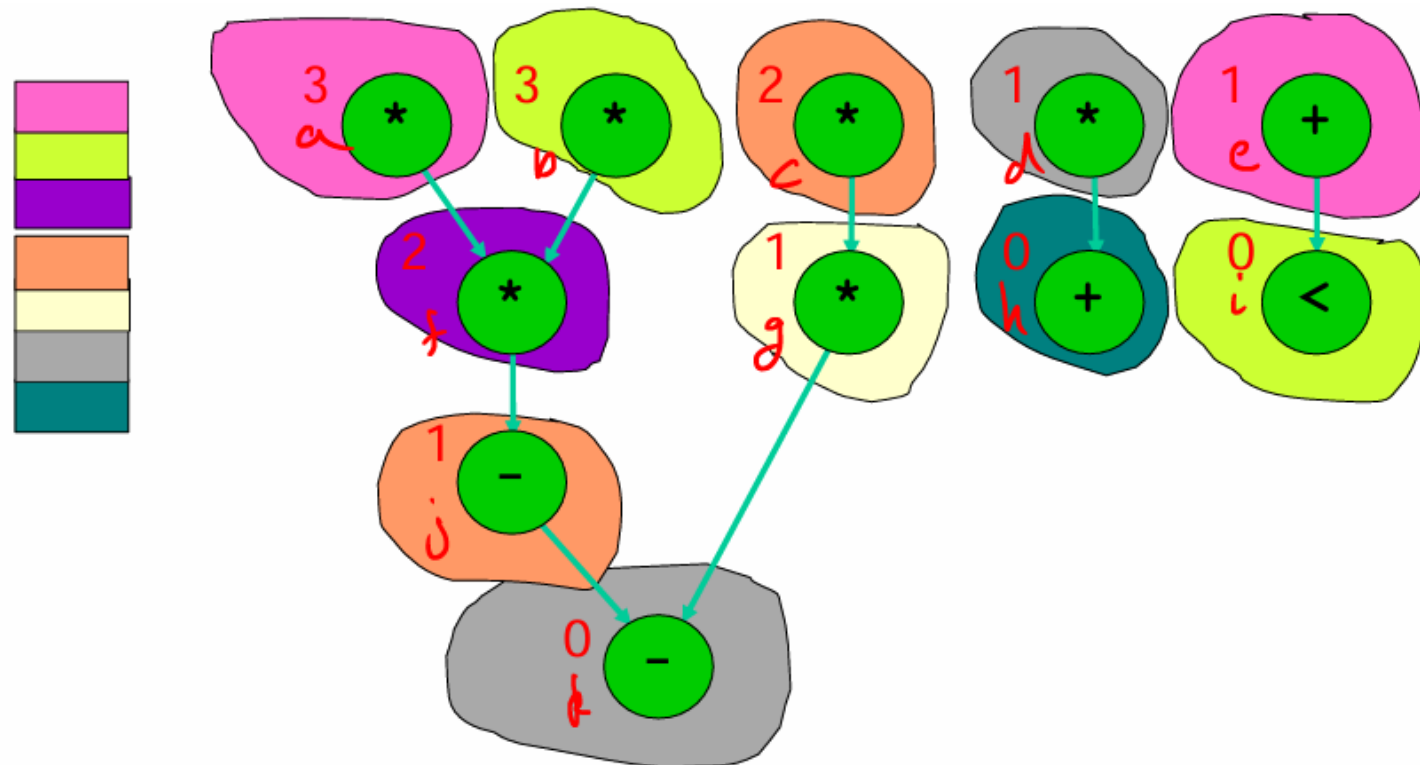
CONSTRAINED SCHEDULING

- List scheduling:
 - o Sort nodes in topological order
 - o Assign priority to nodes
 - o Criteria can be:
 - number of successors,
 - depth (length of longest path from inputs),
 - latency-weighted depth, – w : latency of the operation to be executed by the nodes on the path.
 - mobility,
 - connectivity, • ...

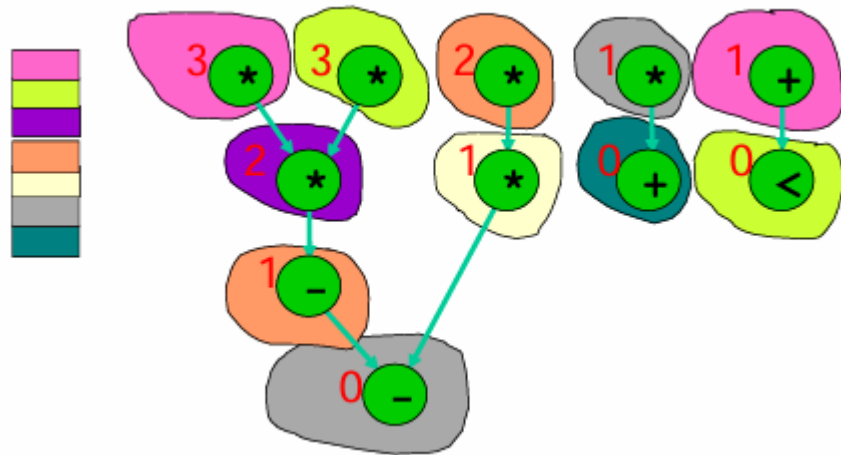


CONSTRAINED SCHEDULING

- Resources: 1 multiplier, 1 ALU (+, -, <)



TEMPORAL PARTITIONING



- **Criterion:** number of successors
- **size(FPGA)** = 250,
- **size (mult)** = 100,
- **size(add)** = **size(sub)** = 20,
- **size(comp)** = 10.

